

Assignment - 1A

Title: Design and implement Parallel Breadth First Search algorithm using OpenMP.

Name – Suraj Pawar

Roll No. – 14254

Div – B

Batch – B3

```
#include <iostream>
#include <queue>
#include <omp.h>

using namespace std;

class node {
public:
    node *left, *right;
    int data;
};

class Breadthfs {
public:
    node* insert(node*, int);
    void bfs(node*);
};

node* Breadthfs::insert(node* root, int data) {
    if (!root) {
        root = new node;
        root->left = NULL;
        root->right = NULL;
        root->data = data;
        return root;
    }

    queue<node*> q;
    q.push(root);

    while (!q.empty()) {
        node* temp = q.front();
        q.pop();

        if (temp->left == NULL) {
            temp->left = new node;
            temp->left->left = NULL;
            temp->left->right = NULL;
            temp->left->data = data;
            return root;
        } else {
            q.push(temp->left);
        }
    }
}
```

```

    }

    if (temp->right == NULL) {
        temp->right = new node;
        temp->right->left = NULL;
        temp->right->right = NULL;
        temp->right->data = data;
        return root;
    } else {
        q.push(temp->right);
    }
}
return root;
}

void Breadthfs::bfs(node* head) {
    if (!head) return;

    queue<node*> q;
    q.push(head);

    while (!q.empty()) {
        int qSize = q.size();

        #pragma omp parallel for
        for (int i = 0; i < qSize; i++) {
            node* currNode;

            #pragma omp critical
            {
                currNode = q.front();
                q.pop();
                cout << currNode->data << "\t";
            }

            #pragma omp critical
            {
                if (currNode->left)
                    q.push(currNode->left);
                if (currNode->right)
                    q.push(currNode->right);
            }
        }
        cout << endl;
    }
}

int main() {
    Breadthfs b;
    node* root = NULL;
    int data;
    char ans;

```

```

do {
    cout << "Enter data => ";
    cin >> data;
    root = b.insert(root, data);
    cout << "Do you want to insert one more node? (y/n): ";
    cin >> ans;
} while (ans == 'y' || ans == 'Y');

cout << "\nParallel BFS Traversal:\n";
b.bfs(root);

return 0;
}

```

Output –

```

Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
datanalytics1@datanalytics1-OptiPlex-7050:~$ g++ -fopenmp parallel_bfs.cpp -o p
arallel_bfs
datanalytics1@datanalytics1-OptiPlex-7050:~$ ./parallel_bfs
Enter data => 1
Do you want to insert one more node? (y/n): y
Enter data => 2
Do you want to insert one more node? (y/n): y
Enter data => 3
Do you want to insert one more node? (y/n): y
Enter data => 4
Do you want to insert one more node? (y/n): y
Enter data => 5
Do you want to insert one more node? (y/n): n

Parallel BFS Traversal:
1
2      3
4      5

```

Assignment – 1B

Title: Design and implement Parallel Depth First Search algorithm using OpenMP.

Name – Suraj Pawar

Roll No. – 14254

Div – B

Batch – B3

```
#include <iostream>
#include <vector>
#include <stack>
#include <omp.h>

using namespace std;

const int MAX = 100000;
vector<int> graph[MAX];
bool visited[MAX];

void dfs(int start_node) {
    stack<int> s;
    s.push(start_node);

    while (!s.empty()) {
        int curr_node;

        #pragma omp critical
        {
            curr_node = s.top();
            s.pop();
        }

        if (!visited[curr_node]) {
            visited[curr_node] = true;

            #pragma omp critical
            {
                cout << curr_node << " ";
            }

            // Parallel exploration of adjacent nodes
            #pragma omp parallel for
            for (int i = 0; i < graph[curr_node].size(); i++) {
                int adj_node = graph[curr_node][i];
                if (!visited[adj_node]) {
                    #pragma omp critical
                    {
                        s.push(adj_node);
                    }
                }
            }
        }
    }
}
```

```

    }
}
}

int main() {
    int n, m, start_node;

    cout << "Enter No of Nodes, Edges and Start Node: ";
    cin >> n >> m >> start_node;

    cout << "Enter Pair of Edges:" << endl;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;

        graph[u].push_back(v);
        graph[v].push_back(u); // Undirected graph
    }

    // Initialize visited array in parallel
    #pragma omp parallel for
    for (int i = 0; i < n; i++) {
        visited[i] = false;
    }

    cout << "\nParallel DFS Traversal:\n";
    dfs(start_node);

    return 0;
}

```

Output –

```

datanalytics1@datanalytics1-OptiPlex-7050:~$ ./parallel_dfs
Enter No of Nodes, Edges and Start Node: 6 5 0
Enter Pair of Edges:
0 1
0 2
1 3
3 4
3 5

Parallel DFS Traversal:
0 1 3 5 4 2 datanalytics1@datanalytics1-OptiPlex-7050:~$

```

Assignment - 2A

Title: Write a program to implement Parallel Bubble Sort.

Name – Suraj Pawar

Roll No. – 14254

Div – B

Batch – B3

```
#include <iostream>
#include <omp.h>

using namespace std;

void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}

void parallelBubbleSort(int *a, int n) {
    for (int i = 0; i < n; i++) {

        int first = i % 2; // 0 for even phase, 1 for odd phase

        #pragma omp parallel for shared(a, first)
        for (int j = first; j < n - 1; j += 2) {
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
            }
        }
    }
}

int main() {
    int n;
    int *a;

    cout << "Enter total number of elements: ";
    cin >> n;

    a = new int[n];

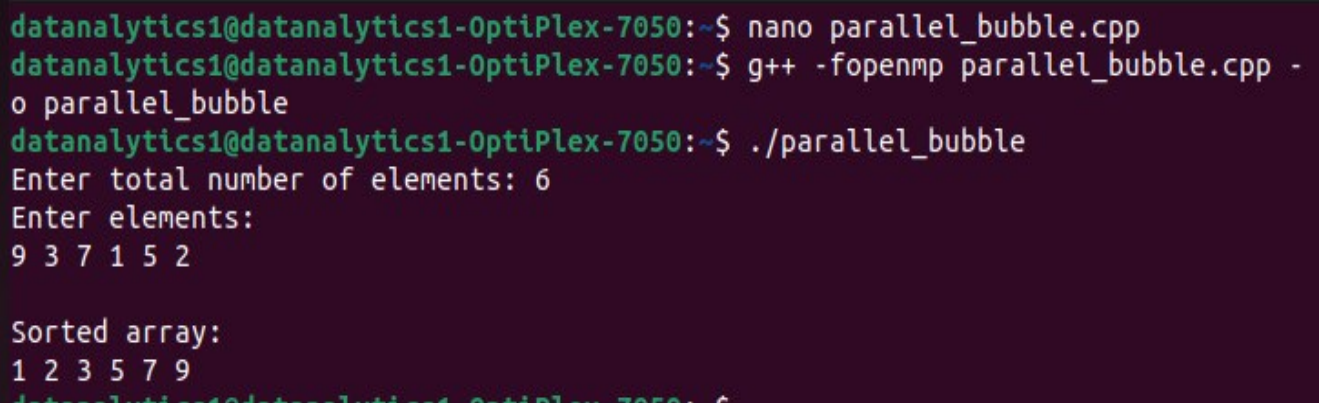
    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    parallelBubbleSort(a, n);

    cout << "\nSorted array:\n";
```

```
for (int i = 0; i < n; i++) {  
    cout << a[i] << " ";  
}  
cout << endl;  
  
delete[] a;  
return 0;  
}
```

Output –



```
datanalytics1@datanalytics1-OptiPlex-7050:~$ nano parallel_bubble.cpp  
datanalytics1@datanalytics1-OptiPlex-7050:~$ g++ -fopenmp parallel_bubble.cpp -  
o parallel_bubble  
datanalytics1@datanalytics1-OptiPlex-7050:~$ ./parallel_bubble  
Enter total number of elements: 6  
Enter elements:  
9 3 7 1 5 2  
  
Sorted array:  
1 2 3 5 7 9  
datanalytics1@datanalytics1-OptiPlex-7050:~$
```

Assignment – 2B

Title: Write a program to implement Parallel Merge Sort.

Name – Suraj Pawar

Roll No. – 14254

Div – B

Batch – B3

```
#include <iostream>
#include <omp.h>

using namespace std;

void merge(int a[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = a[l + i];
    for (j = 0; j < n2; j++)
        R[j] = a[m + 1 + j];

    i = 0;
    j = 0;
    k = l;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            a[k++] = L[i++];
        else
            a[k++] = R[j++];
    }

    while (i < n1)
        a[k++] = L[i++];

    while (j < n2)
        a[k++] = R[j++];
}

void mergeSort(int a[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;

        #pragma omp parallel sections
        {
            #pragma omp section
```



```

        {
            mergeSort(a, l, m);
        }

        #pragma omp section
        {
            mergeSort(a, m + 1, r);
        }
    }

    merge(a, l, m, r);
}

int main() {
    int n;
    cout << "Enter total number of elements: ";
    cin >> n;

    int *a = new int[n];

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    double start, stop;

    start = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp single
        {
            mergeSort(a, 0, n - 1);
        }
    }

    stop = omp_get_wtime();

    cout << "\nSorted array:\n";
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }

    cout << "\nExecution Time: " << stop - start << " seconds\n";

    delete[] a;

    return 0;
}

```

Output –

```
datanalytics1@datanalytics1-OptiPlex-7050:~$ nano parallel_mergesort.cpp
datanalytics1@datanalytics1-OptiPlex-7050:~$ g++ -fopenmp parallel_mergesort.cpp -o parallel_mergesort
datanalytics1@datanalytics1-OptiPlex-7050:~$ ./parallel_mergesort
Enter total number of elements: 6
Enter elements:
12 11 13 5 6 7

Sorted array:
5 6 7 11 12 13
Execution Time: 0.00572774 seconds
datanalytics1@datanalytics1-OptiPlex-7050:~$
```

Assignment 3

Title: Implement Min, Max, Sum and Average operations using Parallel Reduction.

Name: Suraj Pawar

Roll no:14254

Div:B

Batch:B3

```
#include <iostream>
// #include <vector>
#include <omp.h>
#include <climits>
using namespace std;

void min_reduction(int arr[], int n) {
    int min_value = INT_MAX;
    #pragma omp parallel for reduction(min: min_value)
    for (int i = 0; i < n; i++) {
        if (arr[i] < min_value) {
            min_value = arr[i];
        }
    }
    cout << "Minimum value: " << min_value << endl;
}

void max_reduction(int arr[], int n) {
    int max_value = INT_MIN;
    #pragma omp parallel for reduction(max: max_value)
    for (int i = 0; i < n; i++) {
        if (arr[i] > max_value) {
            max_value = arr[i];
        }
    }
    cout << "Maximum value: " << max_value << endl;
}

void sum_reduction(int arr[], int n) {
    int sum = 0;
    #pragma omp parallel for reduction(+: sum)
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    cout << "Sum: " << sum << endl;
}

void average_reduction(int arr[], int n) {
```

```

int sum = 0;
#pragma omp parallel for reduction(+: sum)
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
cout << "Average: " << (double)sum / (n-1) << endl;
}

int main() {
    int *arr,n;
    cout<<"\n enter total no of elements=>";
    cin>>n;
    arr=new int[n];
    cout<<"\n enter elements=>";
    for(int i=0;i<n;i++)
    {
        cin>>arr[i];
    }

    // int arr[] = {5, 2, 9, 1, 7, 6, 8, 3, 4};
    // int n = size(arr);

    min_reduction(arr, n);
    max_reduction(arr, n);
    sum_reduction(arr, n);
    average_reduction(arr, n);
}

```

Output:

```

datanalytics2@datanalytics2-OptiPlex-3050:~$ g++ -fopenmp Ass3.cpp -o output
datanalytics2@datanalytics2-OptiPlex-3050:~$ ./output
Enter total number of elements: 5
Enter elements:
10 20 30 40 50

Results using Parallel Reduction:
Minimum value: 10
Maximum value: 50
Sum: 150
Average: 30
datanalytics2@datanalytics2-OptiPlex-3050:~$

```

Assignment 4(a)

Title: Write a CUDA program for Addition of two large vectors.

Name: Suraj Pawar

Roll no:14254

Div: B

Batch: B3

```
#include <iostream>
#include <cuda_runtime.h>
#include /usr/local/cuda/include/cuda_runtime.h

__global__ void addVectors(int* A, int* B, int* C, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
    {
        C[i] = A[i] + B[i];
    }
}

int main()
{
    int n = 1000000;
    int* A, * B, * C;
    int size = n * sizeof(int);

    // Allocate memory on the host
    cudaMallocHost(&A, size);
    cudaMallocHost(&B, size);
    cudaMallocHost(&C, size);

    // Initialize the vectors
    for (int i = 0; i < n; i++)
    {
        A[i] = i;
        B[i] = i * 2;
    }

    // Allocate memory on the device
    int* dev_A, * dev_B, * dev_C;
    cudaMalloc(&dev_A, size);
    cudaMalloc(&dev_B, size);
    cudaMalloc(&dev_C, size);
```

```

// Copy data from host to device
cudaMemcpy(dev_A, A, size, cudaMemcpyHostToDevice);

cudaMemcpy(dev_B, B, size, cudaMemcpyHostToDevice);

// Launch the kernel
int blockSize = 256;
int numBlocks = (n + blockSize - 1) / blockSize;
// Copy data from device to host
cudaMemcpy(C, dev_C, size, cudaMemcpyDeviceToHost);

// Print the results
for (int i = 0; i < 10; i++)
{
    cout << C[i] << " ";
}
cout << endl;

// Free memory
cudaFree(dev_A);
cudaFree(dev_B);
cudaFree(dev_C);
cudaFreeHost(A);
cudaFreeHost(B);
cudaFreeHost(C);

return 0;
}

```

Output:

```

datanalytics2@datanalytics2-OptiPlex-3050:~$ nvcc vector_add.cu -o vector_add
datanalytics2@datanalytics2-OptiPlex-3050:~$ ./vector_add
0 3 6 9 12 15 18 21 24 27
datanalytics2@datanalytics2-OptiPlex-3050:~$

```

Assignment 4(b)

Title: Write a program for matrix multiplication using CUDA

Name: Suraj Pawar

Roll no:14254

Div:B

Batch:B3

```
#include <cuda_runtime.h>
#include <iostream>
__global__ void matmul(int* A, int* B, int* C, int N) {
    int Row = blockIdx.y*blockDim.y+threadIdx.y;
    int Col = blockIdx.x*blockDim.x+threadIdx.x;
    if (Row < N && Col < N) {
        int Pvalue = 0;
        for (int k = 0; k < N; k++) {
            Pvalue += A[Row*N+k] * B[k*N+Col];
        }
        C[Row*N+Col] = Pvalue;
    }
}

int main() {
    int N = 512;
    int size = N * N * sizeof(int);
    int* A, * B, * C;
    int* dev_A, * dev_B, * dev_C;
    cudaMallocHost(&A, size);
    cudaMallocHost(&B, size);
    cudaMallocHost(&C, size);
    cudaMalloc(&dev_A, size);
    cudaMalloc(&dev_B, size);
    cudaMalloc(&dev_C, size);
    // Initialize matrices A and B
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i*N+j] = i*N+j;
            B[i*N+j] = j*N+i;
        }
    }
    cudaMemcpy(dev_A, A, size,
               cudaMemcpyHostToDevice);
    cudaMemcpy(dev_B, B, size,
               cudaMemcpyHostToDevice);
    dim3 dimBlock(16, 16);
    dim3 dimGrid(N/dimBlock.x, N/dimBlock.y);
```

```

matmul<<<dimGrid, dimBlock>>>(dev_A, dev_B,
dev_C, N);
cudaMemcpy(C, dev_C
// Print the result
for (int i = 0; i < 10; i++) {
for (int j = 0; j < 10; j++) {
std::cout << C[i*N+j] << " ";
}
std::cout << std::endl;
}
// Free memory
cudaFree(dev_A);
cudaFree(dev_B);
cudaFree(dev_C);
cudaFreeHost(A);
cudaFreeHost(B);
cudaFreeHost(C);
return 0;

```

Output:

```

datanalytics2@datanalytics2-OptiPlex-3050:~$ nvcc matrix_mul.cu -o matrix_mul
datanalytics2@datanalytics2-OptiPlex-3050:~$ ./matrix_mul
0 512 1024 1536 2048 2560 3072 3584 4096 4608
512 1024 1536 2048 2560 3072 3584 4096 4608 5120
1024 1536 2048 2560 3072 3584 4096 4608 5120 5632
1536 2048 2560 3072 3584 4096 4608 5120 5632 6144
2048 2560 3072 3584 4096 4608 5120 5632 6144 6656
2560 3072 3584 4096 4608 5120 5632 6144 6656 7168
3072 3584 4096 4608 5120 5632 6144 6656 7168 7680
3584 4096 4608 5120 5632 6144 6656 7168 7680 8192
4096 4608 5120 5632 6144 6656 7168 7680 8192 8704
4608 5120 5632 6144 6656 7168 7680 8192 8704 9216
datanalytics2@datanalytics2-OptiPlex-3050:~$

```


Assignment No.5 - Mini Project

Name: Suraj Pawar

Roll No. 14254

Div : B

Batch : B3

```
In [1]: import tensorflow as tf
        from tensorflow import keras
        import numpy as np
        import time
```

```
In [2]: # Step 1: Start time (to measure performance)
        start_time = time.time()
```

```
In [3]: # Step 2: Create distributed strategy (HPC concept)
        strategy = tf.distribute.MirroredStrategy()
        print("Number of devices used:", strategy.num_replicas_in_sync)
```

INFO:tensorflow:Using MirroredStrategy with devices ('/job:localhost/replica:0/task:0/device:CPU:0',)

Number of devices used: 1

```
In [4]: # Step 3: Load dataset (MNIST)
        (x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 3s 0us/step

```
In [5]: # Step 4: Normalize data
        x_train = x_train / 255.0
        x_test = x_test / 255.0
```

```
In [6]: # Step 5: Build model inside strategy scope (important for HPC)
        with strategy.scope():
            model = keras.Sequential([
                keras.layers.Flatten(input_shape=(28, 28)),
                keras.layers.Dense(128, activation='relu'),
                keras.layers.Dropout(0.2),
                keras.layers.Dense(10, activation='softmax')
            ])

            # Compile model
            model.compile(
                optimizer='adam',
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy']
            )
```

C:\Users\pawar\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

```
In [7]: # Step 6: Train model (parallel execution)
        history = model.fit(
            x_train, y_train,
            epochs=5,
            batch_size=64,
            validation_split=0.1
        )
```

WARNING:tensorflow:From C:\Users\pawar\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\backend\tensorflow\core.py:232: The name tf.placeholder is deprecated. Please use tf.compat.v1.placeholder instead.

Epoch 1/5

844/844  2s 2ms/step - accuracy: 0.8964 - loss: 0.3580 - val_accuracy: 0.9612 - val_loss: 0.1448

Epoch 2/5

844/844  2s 2ms/step - accuracy: 0.9484 - loss: 0.1761 - val_accuracy: 0.9685 - val_loss: 0.1068

Epoch 3/5

844/844  2s 2ms/step - accuracy: 0.9615 - loss: 0.1299 - val_accuracy: 0.9718 - val_loss: 0.0990

Epoch 4/5

844/844  2s 2ms/step - accuracy: 0.9685 - loss: 0.1055 - val_accuracy: 0.9778 - val_loss: 0.0826

Epoch 5/5

844/844  2s 2ms/step - accuracy: 0.9722 - loss: 0.0887 - val_accuracy: 0.9775 - val_loss: 0.0765

```
In [8]: # Step 7: Evaluate model
        loss, accuracy = model.evaluate(x_test, y_test)
```

313/313  0s 1ms/step - accuracy: 0.9763 - loss: 0.0785

```
In [9]: # Step 8: End time
        end_time = time.time()
```

```
In [10]: # Step 9: Print results
          print("\nTest Accuracy:", accuracy)
          print("Total Training Time:", round(end_time - start_time, 2), "seconds")
```

Test Accuracy: 0.9763000011444092

Total Training Time: 127.29 seconds

```
In [ ]:
```

Assignment No:5

Title of the Assignment: Implementation of High Performance Computing (HPC) Application for AI/ML using TensorFlow Distributed Training

Objective of the Assignment: To implement a high-performance computing (HPC) based AI/ML model using parallel processing techniques to improve training speed and efficiency.

Theory:

High Performance Computing (HPC) refers to the use of powerful computing resources such as multi-core processors, GPUs, and distributed systems to solve complex computational problems efficiently. In the domain of Artificial Intelligence and Machine Learning, HPC plays a crucial role in handling large datasets and training complex models within a reasonable time.

HPC systems use **parallel computing**, where multiple computations are performed simultaneously. This is achieved using techniques such as:

- **Data Parallelism:** Splitting data across multiple processors and performing the same operation on each subset.
- **Model Parallelism:** Dividing the model across different processors for simultaneous computation.

In AI/ML, training deep learning models involves large matrix operations and iterative computations, which are computationally expensive. HPC accelerates this process using:

- **GPU Acceleration:** GPUs perform thousands of parallel operations, making them ideal for deep learning tasks.
- **Distributed Computing:** Multiple machines (nodes) work together to train a model faster.
- **Batch Processing:** Data is processed in batches to optimize memory usage and speed.

Frameworks like **TensorFlow** and **Keras** provide built-in support for HPC through distributed training strategies such as:

- `MirroredStrategy()` for single-machine multi-GPU training
- `MultiWorkerMirroredStrategy()` for multi-node distributed training

These strategies replicate the model across devices and synchronize updates, enabling efficient parallel training.

HPC also improves:

- **Scalability:** Ability to handle increasing data size and model complexity
- **Performance:** Faster training and reduced execution time
- **Resource Utilization:** Efficient use of CPU/GPU resources

However, HPC systems may require specialized hardware, high setup cost, and technical expertise to manage distributed environments.

Algorithm

1. Load dataset (MNIST)
2. Normalize input data
3. Initialize distributed strategy
4. Build model inside strategy scope
5. Compile model
6. Train model using parallel execution
7. Evaluate performance

Conclusion

Parallel computing in HPC significantly enhances the performance of AI/ML models by utilizing multiple processing units. This approach is essential for handling large-scale machine learning problems.